CMSI 3802

Final Exam Preparation

The best way to prepare for your final is to:

- Know all the logistical details
- Make sure the learning objectives of the course were satisfied
- Make sure you have learned the essentials
- Review the course in outline (or story) form
- Understand the types of questions will appear on the exam
- Quiz yourself and your friends (recall >>>>>> rereading)

Logistics

You will take the final on BrightSpace during the allotted time for the exam, which is Monday, May 12, 2025, from 11:00 to 1:00pm America/Los Angeles time. You will have 120 minutes to do the exam. All problems will be multiple choice, multiple select, or matching. Submit on time; the penalty for late submissions is 5 points per minute.

Expect 20-30 questions. There will be *no* nasty all-of-the-above or none-of-the-above options. There will, however, occasionally be questions for which one answer is solid and correct, while others may make true—or nearly-always-true—statements but do not actually fit 100% with the asked question.

The intent of the questions is not to trick you. The intent is to assess whether you have gained a fairly deep understanding of computer science fundamentals (linguistic and theoretical concepts) without having to do a one-hour web search. All students can be this kind of student. If you have read the materials (and there was indeed a ton of reading!) and have participated in the writing of your term project (a real compiler!), you *are* that kind of student.

As always, you **MAY** use books, notes, and web *searches* to look things up. You will not be spied on: there is no browser lock down and hence no need to hide a mobile device in a bag of potato chips. However, you **MAY NOT** solicit answers in any way. There is to be no asking for help, no posting on forums, no communication with other humans or chat bots in any way; you can only “look things

up," you may never "ask." You also **MAY NOT** post answers or help any other test taker either. You are bound by an honor code to follow these rules.

Learning Objectives Review

Review the learning objectives from the syllabus now. If there is any unmet objective, let the instructor know. For reference, the objectives are repeated here:

- Gain a working knowledge of two major theories of computation—language theory and automata theory—and a brief familiarity with two others—computability theory and complexity theory;
- Become accomplished in language processing techniques, compiler theory, compiler design techniques, and the notion of intermediate languages and virtual machines;
- Increase their software development expertise by building a complex system (a compiler) using a modern tool set including Node, Ohm, and c8, as an open source project hosted on a public repository; and
- Be able to tell the story of the birth and subsequent evolution of Computer Science.

The Essentials

There are several things that all recipients of a computer science degree are expected to know. Ideally, these would be verified by an oral exam with a pass/fail outcome. Don't worry, such an oral exam is not feasible, and may even be subject to the examinee failing due to nerves, so you're going to test yourself in the comfort of your own space. Make sure you know the following:

- What are the four major theories of computation?
- What is language theory concerned with?
- What is automata theory concerned with?
- What is computation theory concerned with?
- What is complexity theory concerned with?
- What is a formal language?
- What is a grammar?
- What is the difference between generating and recognizing a language?
- Who is the person most associated with classifying kinds of grammars by their expressive power?
- Why was the Turing Machine invented?

- What are Turing's main contributions?
- How does a Turing Machine work?
- What kind of languages are recognized by (general) Turing Machines?
- What kind of languages are recognized by Linear Bounded Automata (LBAs)?
- What kind of languages are recognized by Pushdown Automata (PDAs)?
- What kind of languages are recognized by Finite Automata (FAs)?
- What is the difference between recognition and decidability?
- Is the set of all TMs that halt recognizable? decidable?
- What does the Church-Turing Thesis say?
- What is the main evidence for the Church-Turing Thesis?
- What characterizes context-free grammars?
- What is a regular expression?
- What is a compiler?
- What is an interpreter?
- What is a virtual machine?
- What is Ohm?
- What happens in the front-end and in the back-end of a compiler?
- What are the three phases of analysis in the compiler?
- What happens in lexical analysis?
- What happens in syntax analysis?
- What happens in semantic analysis?
- What happens in code generation?
- How do concrete syntax trees differ from abstract syntax trees?
- Why is syntax analysis "easy" and semantic analysis "hard"?

Do you know how to answer each and every question? Can you articulate good answers to each? If so, great! Congratulations! This is the minimal criteria for passing.

Practical Learnings!

You learned some theory and some bits of knowledge. But hopefully, you got so much more. You should have:

- Become proficient in building large apps in Node with NPM, c8, etc.
- Improved your skills in *modern* JavaScript
- Learned what it takes to implement a language
- Become a much better programmer than you were
- Gotten really good at regular expressions

- Mastered the description of syntax with Ohm
- Learned the difference between syntax and semantics and the connections between them
- Learned the various phases of language translation
- Gotten experience writing code that essentially traversed and transformed large trees and graphs
- Become acquainted with a number of optimization strategies and tricks
- Felt good about actually seeing programs written in a language of your own design actually run!

Why did you write a compiler?

- Writing a compiler helps you understand the compilers you use every day;
- Most programmers actually do write translators: most apps have some data that's stored in a "config file," or you may encode data structures in a string here and there and have to parse them;
- Knowing what compilers do and how they work can help you write more efficient code since you know what a compiler is capable and not capable of optimizing; and
- A compiler makes an awesome capstone project.

Course Notes Review

Back to the academic side of things.

Review the course notes covered during lectures.

Course Story

CMSI 3801 and 3802 enable you to tell the story of the discipline of computer science and thus prepare you to succeed as a practitioner in the field and even make theoretical contributions to the field. We can understand this story roughly as follows:

- Humans need **information** to survive.
- We must both *store* and **process** information.
- The processing of information, often generating new information, by mechanical means, is called **computation**.
- Throughout history, humans have done computation verbally and in writing, and eventually created many mechanical computing devices.

- After thousands of years of computing, humanity was able to put computation itself on a formal footing: the Lambda Calculus and Turing Machines were invented for this purpose.
- During the formalization of computing, the limits of computation were discovered. Computability theory arose to show what could and could not be computed. The **Halting Problem** is the most famous problem associated with this theory.
- The "limits of computation" thing is cool and all, but the most astounding and impactful result of formalizing computation was the discovery of **computational universality**. This was hinted at in the 19th writings of Lovelace, but made precise by Church and Turing.
- Once universal computation was understood, computers with power unimaginable to early 20th century humans started to appear.
- While the Lambda Calculus and Turing Machines were useful to express what computing is, and what computers could do, they lacked power. So register machines were invented. Physical machines based on the register model have been improving ever since.
- Despite the complexity of modern CPUs, GPUs, and TPUs, they can be understood. It is good to learn machine and assembly language.
- Rather than programming directly in machine or assembly language, humans have invented **virtual machines**, that are easy to interpret and that map pretty easily to real machines. A good virtual machine can have implementations of many different physical machines. (This mapping of virtual to physical is one of the most important themes of computer science.)
- Even virtual machines are hard for humans to express complex computation, for that we need high level languages. The early HLL trailblazers were Backus, Hopper, and McCarthy.
- High level languages need to be translated (compiled) into virtual machine code (for interpretation) or, in theory, translated all the way to assembly language. We usually target virtual machines in a real compiler (you should know why).
- High level languages bring with them many academic and engineering challenges. The first HLLs were given compilers that were massive hacks. Then, the linguists and computing researchers got involved, and **language theory** was created.
- Generative grammars were created first as a way of formally describing the syntax of a language; then came analytic grammars to drastically simplify compiler writing.
- Real programming languages require much, much, more than the basic results of language theory. A real syntax distinguishes tokens from phrases, requires a lot of punctuation, distinguishes keywords from identifiers, must support comments, and deal with operator precedence, associativity, fixity, and arity.
- Language theory and advances in understanding syntax have allowed **parsers** to be automatically generated from an analytic grammar. Ohm is a library that can do this for you.
- A syntax is not sufficient to define a language. Semantics is required, too. Semantics is somewhat messier than syntax. All languages can be defined with the same grammar notation, but for semantics there are a lot of seemingly ad-hoc rules.

- Semantics encompasses both statics (things that can be checked before running the program) and dynamics (the effect of running the program).
- The best way to understand language definition and language processing is to write your own compiler.
- Writing a compiler is aided by understanding not just formal syntax but **abstract syntax**. And by not just generating good code but **efficient** code.
- Code optimization (a.k.a. code improvement) is a fascinating field of study. The biggest name in this area is Frances Allen.

> *The field of theoretical computer science, and its connections to programming language definition, usage, and implementation, is so much broader than what can be fully grasped in one year of college study. You've been given a taste of the field, thrown into writing a compiler, and challenged with difficult questions without much preparation other than "hopefully you did all the readings and gain some insight form what you read". React to these experiences with a desire to learn those things beyond your comfort zone, and not with a depressive feeling that you should have been able to do every assignment or learn every concept.That is not how the world is.*

Course Outline

Here is a rough outline of the course material. The outline is not exactly in the order the material was presented during lecture, since the optimal ordering of learning is not the same as the overall outline of a subject.

Note that because our course was only one semester, certain things were not covered, and are ~~so marked with the strikeout decoration~~.

```
THEORIES OF COMPUTER SCIENCE
    What is a theory?
    Why do we have theories?
    Historical path to computer science as a discipline
        Information
        Computation in Antiquity (early recipes)
        Early "Machines"
        Mathematics and Philosophy
        Formalization of Computation
        Turing
        Electronic Computers
        Programming Languages
        Compilers
```

- Optimization
- Computing for People
- The four major theories of computation and their concerns
  - Language Theory
    - Concerned with how computations are expressed
  - Automata Theory
    - Concerned with how computations are performed
    - Sneak peek: Turing Machines
    - The stunning notion of computational universality
  - Computation Theory
    - Concerned with what can and cannot be computed
    - Sneak peek: the halting problem is undecidable
  - Complexity Theory
    - Concerned with how efficiently computations can be performed
    - Sneak peek: P vs. NP

LANGUAGE THEORY

- Concerned with how computations are expressed
- Why study language theory?
- Information representation
- Formal Language Theory
  - Symbols, Alphabets, Strings, Languages
  - Operators on languages: Union, Intersection, Concatenation, Kleene Star
  - How to formally define a language?
  - Generative Grammars
    - Role of variables
    - Grammar notation
    - How strings are generated
    - Lots of example grammars
    - Parse Trees (aka Derivation Trees)
    - Ambiguity
    - Formal Definition (won't be on the exam)
    - Restrictions
      - CFG: LHS is only one variable
      - RLG: LHS is only one variable and RHS is symbols + at most one variable
      - ENG: Never shrinks (except you can have the rule s->ε)
  - Language Recognition
    - Automata can be used for this
    - Analytic Grammars
  - Language Classification
    - Chomsky Hierarchy for Formal Languages (original version)
      - Regular (Type 3)
      - Context-Free (Type 2)
      - Type-1 (aka "Context sensitive")
      - Unrestricted (Type 0)
    - Larger Chomsky Hierarchy
      - Finite
      - Regular
      - Context-Free

- “Context-Sensitive”
- Recursive
- Recursively Enumerable (r.e.)
- Finitely Describable

- Programming Language Theory
  - How PLT differs from formal language theory
  - Concerns (Just a list for now)
    - Syntax
    - Semantics
    - Type Theory
    - Static Analysis
    - Translation
    - Runtime Systems
    - Verification
    - Metaprogramming
    - Classification

SYNTAX

- Motivation (there is a structure underlying all programs)
- Many ways to express this structure as a string
- Definition of Syntax
- Syntax Diagrams
- Lexical vs. Phrase Syntax
  - Why this is massively important
  - Ways to represent the difference
- Tokens
- Parse Trees
  - The frontier of the parse tree is the token stream
- Dealing with Ambiguity
  - Precedence (and how to capture it in a grammar)
  - Associativity (and how to capture it in a grammar)
- Parsing (sneak peek only)
  - Hand-crafted, recursive descent
  - Parser generators
  - Analytic Grammars
  - PEGs
  - Ohm
- The Problem of Context
  - Things you cannot capture in a context-free grammar, incomplete list:
    - No redeclare within scope
    - No use of possibly uninitialized variables
    - Type checking
    - Correct number of arguments must appear in a call
    - Access modifiers must be correct
    - All execution paths through a function must end in a return
    - All abstract methods must be implemented or declared abstract
    - All declared local variables must be used
    - All private methods in a class must be used
  - Is this stuff syntax or semantics?

        People can disagree
    Side note: can be formalized in theory but why bother
Type inference
Abstract Syntax
    What ASTs look Like
    Difference between CSTs (Parse trees) and ASTs
    Tree grammars to formally define ASTs
    Esprima
    Examples in JavaScript
    Examples in Java
Aside: Different syntax formalisms in the real world

LANGUAGE DESIGN
Things to know
Major features of existing programming languages
Historical Issues
    What Bret Victor says about the 1960s and 1970s
    What Alan Kay thinks
The process of language design
    Big picture and big questions
    Starter set of features
    Design your abstract syntax
    Sketch and Prototype with Ohm!!!
    Start working on lower-level syntax
    What kind of sugar do you want?
Differences between syntax, semantics, pragmatics
Ohm for language design
    Ohm grammar notation
    Ohm details
    Examples of Ohm grammars
Case study: Astro
Case study: Bella
Case study: Carlos

COMPILERS
Translators vs. interpreters
Compilers, assemblers, transpilers
AOT vs. JIT
Overall structure of translation
    Analysis -> Generation
    Analysis -> Optimization -> Generation
    Parsing -> Static Analysis -> Optimization -> Code Generation
    Lexical Analysis
        characters to tokens
    Syntax Analysis = Parsing
        Tokens to CSTs
    Semantic Analysis = Static Analysis
        CSTs to ASTs
        ASTs are pretty much DAGs, so best to call them program representations

- Type checking and other semantic analysis
  - Storing types with expressions in the DAG nodes
  - Reps have nodes not in the AST, e.g. actual functions and variables
- Design decisions: can we simplify the storage of numbers and types?

Intermediate Representations

- Why have them?
- ~~Sneak peek: later phases of the compiler~~
- Control Flow Analysis
- Data Flow Analysis
- Optimization of decorated AST
- Production of high-level language code
- Production of abstract intermediate structures
- Production of bytecode
- Production of abstract assembly language
- Machine independent optimization

Modern compilers are not just one-shot translators

How to architect a compiler using Ohm

- parser.js
- analyzer.js
  - Representing context
  - Checks, especially type checking
- optimizer.js
- generator.js
- core.js
- compiler.js
- <your-language-name>.js
- Tests for compiler, parser, analyzer, optimizer, generator

Why you should write a compiler

AUTOMATA THEORY

Concerned with how computations can be carried out

Broad classification of automata

- Transducers vs Recognizers/Deciders
- Tapes vs Registers
- State Machines vs Instruction Lists
- Harvard vs von Neumann Architecture

Turing Machines

- How they work
- Many Examples
- Variations that neither restrict nor expand computing power
  - Multi-track
  - Multi-head
  - Multi-tape
  - Queue
- Variations that restrict computing power
  - LBAs: Bounded tape
  - PDAs: Input is read-only, read left-to-right once, memory is a stack
  - FAs: Input is read-only, read left-to-right once, no memory

Register Machines

- Counter machines
- RAMs

Other “Automata-like” Formalisms

- String rewriting systems
- λ-Calculus
- Brainf**k
- Recursive Functions (not covered in class)

~~Applications to Intermediate Representations~~

- Why have them?
  - Analysis/Synthesis is inherent to translation
  - Break down complex problem
  - Retargetability
  - For machine independent optimizations
- High-level vs. Medium-level vs. Low-level
- Styles
  - Abstract assembly language (instructions called tuples)
  - Stack code
- List of well-known IRs
  - JVM
  - CLR
  - LLVM
  - SIL
  - CIL
- Tuples

~~Applications to Virtual Machines and Real Machines~~

- Machine Architecture
- How machines work (review)
- Intel 64 architecture
- Review of x86-64 Assembly Language
  - Registers and instructions
  - Calling conventions
  - Parallel instructions
- ARM 64 (AArch64)
  - Registers and instructions
  - Calling conventions
  - Parallel instructions
- The nastiness of conditional jumps and how to avoid them

Code Generation

- Goals
- Translation to JavaScript
- ~~Translation to Assembly Language~~
  - Naïve
  - Interpretive
  - Code generator generators
- ~~Generation of real assembly language~~
  - Address assignment
  - Instruction selection
  - Register allocation
  - Low-level optimization

- ~~Understanding the runtime system for block-structured languages~~
    - Stack frames
    - Dynamic links
    - Static links
    - Register save area
    - Register spilling

PARSING THEORY
- What is parsing?
- Lexical vs Syntactic parsing
- Regular expressions
    - In theory (type-3)
    - In practice
        - Common notation for Regexes in modern languages
        - ( ) [ ] { } ^ $. \ ? * + |
        - Uses: validation, search, extraction, replace
        - Groups
        - Quantifiers
            - Eager: * + ? {}
            - Reluctant: *? +? ?? {}?
            - Possessive: *+ ++ ?+ {}+
        - Backreferences \1 \2 ...
        - Anchors: ^ $ \A \Z \b \B
        - Lookarounds: ?= ?! ?<= ?<!
        - Performance concerns
- ~~Approaches to parsing~~
    - Top-down, LL, Expand-Match
    - Bottom-up, LR, Shift-Reduce
- ~~Recursive Descent~~
- PEGs
- ~~Parsing in the Real world~~

COMPUTABILITY THEORY
- Concerned with what can and cannot be computed
- History: Hilbert, Gödel, Church, Turing
    - Bernhardt book
    - Wadler video
- So many equivalent models, all Turing-complete, hence the Church-Turing Thesis
- There exist noncomputable Functions
    - We show this by diagonalization
- Halting Problem is undecidable
- Limits
    - Non-computable functions = Non-recognizable languages
    - Non-decidable problems = Non-decidable languages
- Reductions
- Rice's Theorem
- Chomsky Hierarchy: The Full Version
    - Finite = S->a|b|c = Non-looping FAs
    - Regular = Right Linear Grammar = Finite Automata

- Deterministic Context Free = LR = DPDA
- Context Free = CFG = (N)PDA
- Type-1 = Linear Bounded Automata
- Decidable (Recursive) = Turing Machines that always Halt
- Recognizable = r.e. = Turing Machines
- Finitely Describable (no machine out here)
- Beyond Finitely Describable 🤯

COMPLEXITY THEORY

- Concerned with how expensive certain computations are
- Time complexity
- Space complexity
- ~~Theory~~
  - Big-O, Big-Theta, Big-Omega
  - Little-O, Little-Theta, Little-Omega
  - Asymptotic Notation
  - P vs. NP
  - NP-Completeness
  - The Complexity Zoo
- Practice: Optimization in Compilers
  - Code Optimization
  - Machine independent vs. machine dependent
  - Constant folding
  - Strength reductions
  - Algebraic simplifications
  - Operand reordering
  - Unreachable code elimination
  - Dead code elimination
  - Copy propagation
  - Common subexpression elimination
  - Loop unrolling
  - ~~Special purpose instructions~~
    - ~~e.g. muladd, range, conditional jump~~
  - ~~Loop invariant factoring~~
  - Tail recursion elimination
  - Induction variable simplification
  - ~~Static frame allocation~~
  - ~~Stack frame simplification~~
  - ~~Low-level optimizations~~
    - Special instructions
    - Alignment
    - Cache
    - Removing conditional jumps
    - Scheduling to remove load delays and similar things

A Skills Check

Here are things you should be able to do before taking the final. Quiz yourself. Quiz each other.

- Make ASTs. Given a fragment of code in some Java-like language, you should be able to draw a tree at exactly the level of detail in the answers I give for homework problems and previous exams.
- Determine whether certain things are lexical errors, syntax errors, static semantic errors, dynamic semantic errors, or not errors.
- Write regular expressions. Everyone loves these.
- Write some Ohm text, or answer questions about why an Ohm attempt is wrong.
- Given some Ohm describing a language, write a program in it.
- Suppose you needed to add new features to the Carlos language. What would the JavaScript for the new entity class look like? Would any optimizations apply to it?
- Explain the types inferred by a given code fragment.
- Identify and apply optimizations, and know when they can and not apply.

Advice for Success on the Exam

You have to put in the time for effortful self-study. Although the exam is open resources, you will not have time to look *everything* up. Those who come in with a strong comfort level with the material will finish on time. I am assessing your fluency and your proficiency with the material, not your Google-Fu.

Bonus: Focus Areas for the Exam Problems

There will be 20 questions. They will be scrambled for each student. The focus of each of the questions are as follows:

- Church-Turing Thesis
- Syntactic Sugar
- Grammar properties (e.g., ambiguity, context-freedom)
- How associativity rules are captured by grammar rules
- Which optimization does this thing

- One of the flags of a regex, might be i, g, m, s
- A strength reduction optimization - is it safe?
- A translation of a Python expression into JavaScript
- Difference between analyzer and optimizer
- That thing I always talk about with generics and subclasses
- How to do analysis on the function composition operator
- Something about how we attach those suffixes during code generation
- Alternative program representations for literal values
- Alternative program representations for types
- ASTs vs Program Represenations
- Which of these regular expressions actually matches this pattern I have in mind
- Is this Java code a lexical, syntax, semantic, or or error?
- Something that translating to JavaScript may or may not do easily
- Which of the following register machine programs is a correct translation
- What the Carlos context is the way it is

Advice for Success in Computer Science

An education is a long-term life journey. Education goes way beyond your chosen field and way beyond academics in general. That said, there is much to be gained by immersing oneself in the history, theory, and practice, of computer science. Our culture is primarily literary, so to that end, you have been assigned a great deal of reading? Were you able to read or skim everything? I hope so, but if not, find time to catch up (or at least please consider catching up in the near future). Among the readings that will be helpful in your journey to becoming a computer scientist, review:

- Turing's Vision by Chris Bernhardt
- Sipser book
- Mogenson book
- Bios on Alonzo Church, Alan Turing, Fran Allen
- Jade's videos on TMs, P vs NP, Russell's Paradox
- Ohm Docs
- Turing's original paper
- Graydon Hoare's presentation
- Alan Turing's Forgotten Ideas
- Ada and the First Computer
- Decoding an Ancient Computer
- The Origins of Computing

- All the assigned Wikipedia articles